ADS205 Tutorial 6 - CNF, CFL Pumping, and Closure Properties

October 19, 2026

1 ADS205 — Decision, Computation and Language

2 Tutorial 6 — Chomsky Normal Form, CFL Pumping, and Closure Properties

Solutions are folded beneath each problem — click the arrow to expand them. Attempt the problem first.

2.1 Reference card

2.1.1 Chomsky normal form

A CFG is in CNF when every rule has one of the forms

$$A \to BC \qquad \text{or} \qquad A \to a,$$

where A, B, C are variables and a is a terminal. The optional rule $S_0 \to \varepsilon$ is allowed when ε belongs to the language. A standard conversion uses a fresh start variable that never appears on a right-hand side.

2.1.2 Pumping lemma for CFLs

If L is context-free, then there exists $p \geq 1$ such that every $s \in L$ with $|s| \geq p$ has a decomposition

$$s = uvxyz$$

satisfying

$$|vy| > 0, \qquad |vxy| \leq p, \qquad uv^i xy^i z \in L \text{ for every } i \geq 0.$$

The pieces v and y are pumped using the **same** exponent.

2.1.3 Selected closure properties

Operation	Regular languages	Context-free languages
Union, concatenation, star	yes	yes
Complement	yes	no
Intersection	yes	no
Intersection with a regular language	yes	yes
Homomorphism	yes	yes

---

3 Part A — CNF and the CFL pumping lemma

3.1 Problem 1 — Convert a grammar to CNF

Check the following CFG

$$S \to ABC, A \to 01, B \to \varepsilon, C \to 11$$

Convert it to Chomsky normal form.

Solution — click to expand

Introduce a new start variable S_0 and let it generate the original start variable $S_0 \to S$

Variable B generate ε, which doesn't affect the generated strings, so directly disgard this rule, so $S \to AC$.

Considering unit rules, we have $S_0 \to S$, so copy the rule of S, i.e., $S \to AC$, and we have $S_0 \to AC$. Since now S doesn't appear at the RHS of the rules concerning S_0, we could get rid of S.

Introduce terminal variables

$$U_0 \to 0, \qquad U_1 \to 1.$$

so $A \to 01$ and $C \to 11$ can be changed to $A \to U_0U_1$ and $C \to U_1U_1$.

To sum up, the CNF form of the given CFG is:

$$\begin{aligned} S_0 &\to AC, \\ A &\to U_0U_1, \\ C &\to U_1U_1, \\ U_0 &\to 0, \\ U_1 &\to 1. \end{aligned}$$

3.2 Problem 2 — Quantifiers and the two pumped pieces

Answer the following.

1. Put the choices in order: pumping length p, witness s, decomposition $uvxyz$, exponent i.
2. Who controls each choice in a proof by contradiction?

3. Can we pump v twice while leaving y unchanged?
4. What does $|vxy| \leq p$ tell us geometrically about a block-structured witness?
5. Why does finding one decomposition that pumps successfully fail to refute a non-CFL proof?

Solution — click to expand

1. Assume L is context-free; the lemma supplies p. The prover chooses $s \in L$ with $|s| \geq p$. An adversary chooses any valid decomposition. The prover then chooses i.
2. A correct non-CFL proof must handle **every** valid decomposition and find an exponent that breaks each one.
3. **No.** Pumping produces uv^ixy^iz, so both pieces use the same exponent.
4. The entire window containing v, x, and y is short. In a witness built from blocks of length p, this often prevents the window from reaching three separated blocks.
5. The lemma promises the existence of some good decomposition for each sufficiently long string. Showing that one selected decomposition works says nothing about the others.

3.3 Problem 3 — Prove $\{a^nb^nc^n\}$ is not context-free

Use the CFL pumping lemma to prove that

$$B = \{a^nb^nc^n \mid n \geq 0\}$$

is not context-free. Your case analysis should explain why vxy cannot touch all three blocks.

Solution — click to expand

Assume that B is context-free and let p be its pumping length. Choose

$$s = a^pb^pc^p.$$

For any valid decomposition $s = uvxyz$, we have $|vxy| \leq p$. Therefore the window vxy cannot contain both an `a` and a `c`: reaching from the `a` block to the `c` block would require crossing all p copies of `b` and would make the window longer than p.

Thus v and y together affect symbols from at most two adjacent blocks. Since $|vy| > 0$, pumping down with $i = 0$ removes at least one symbol from one or two blocks while leaving the third block unchanged. If removal crosses a boundary in a way that disturbs the order $a^*b^*c^*$, the result is already outside B; otherwise the three symbol counts are no longer equal. Hence

$$uxz \notin B.$$

This contradicts the pumping lemma, so B is not context-free.

---

4 Part B — Closure-based proofs

4.1 Problem 4 — CFLs are not closed under intersection or complement

Let

$$A = \{a^m b^n c^n \mid m, n \geq 0\},$$

$$C = \{a^n b^n c^m \mid m, n \geq 0\}.$$

1. Give a CFG, or a short CFG construction, showing that each language is context-free.
2. Compute $A \cap C$.
3. Use Problem 3 to prove that CFLs are not closed under intersection.
4. Use De Morgan's law and the fact that CFLs are closed under union to prove that CFLs are not closed under complement.

Solution — click to expand

For A, use $S \to DE$, $D \to aD \mid \varepsilon$, and $E \to bEc \mid \varepsilon$. The grammar in Problem 4 generates C. Both languages are context-free.

A string in both languages must satisfy the equalities between the `a` and `b` counts and between the `b` and `c` counts. Therefore

$$A \cap C = \{a^n b^n c^n \mid n \geq 0\},$$

which is not context-free by Problem 3. Hence CFLs are not closed under intersection.

Now suppose CFLs were closed under complement. Since they are closed under union, De Morgan's law would give

$$A \cap C = \overline{\overline{A} \cup \overline{C}},$$

and the right-hand side would be context-free. This contradicts the result above. Therefore CFLs are not closed under complement.

4.2 Problem 5 — Filter the copy language

Let

$$D = \{ww \mid w \in \{0,1\}^*\}$$

and let the regular language $R = 0^*1^*0^*1^*$.

1. Compute $D \cap R$.
2. Prove that the resulting language is not context-free using the witness $0^p1^p0^p1^p$.
3. Conclude that D is not context-free.

Solution — click to expand

The regular filter forces each half of a copied string to consist of a `0` block followed by a `1` block. Thus

$$H = D \cap R = \{0^m 1^n 0^m 1^n \mid m, n \geq 0\}.$$

Pumping-lemma proof that H is not context-free. Assume, for a contradiction, that H is context-free. Let $p \geq 1$ be the pumping length promised by the CFL pumping lemma. Choose

$$s = 0^p 1^p 0^p 1^p \in H, \qquad |s| = 4p \geq p.$$

We consider the string as a concatenation of 4 blocks, i.e., 0^p, 1^p, 0^p, and 1^p.

Consider an **arbitrary** decomposition $s = uvxyz$ satisfying

$$|vxy| \leq p, \qquad |vy| > 0.$$

We will show that pumping down, with exponent $i = 0$, gives $uv^0xy^0z = uxz \notin H$ for every such decomposition.

Number the four blocks of s from left to right. The window vxy can meet at most two adjacent blocks: meeting three would require it to contain the entire intervening block of length p and at least one symbol on each side, giving length at least $p+2$. In particular, it cannot meet both blocks 1 and 3, or both blocks 2 and 4.

For $j = 1, 2, 3, 4$, let d_j be the number of symbols pumped down from block j when v and y are pumped down. Then

$$1 \leq d_1 + d_2 + d_3 + d_4 = |v| + |y| \leq p,$$

and the pumped-down string is

$$uxz = 0^{p-d_1} 1^{p-d_2} 0^{p-d_3} 1^{p-d_4}.$$

There are two cases.

Case 1: no block is completely deleted. All four displayed exponents are positive, so these are exactly the four maximal blocks of uxz. At least one d_j is positive. Its corresponding block (1 paired with 3, or 2 paired with 4) is outside the window and loses no symbols. Thus one block has length strictly less than p while its corresponding block still has length p. This violates the equalities required in $0^m 1^n 0^m 1^n$, so $uxz \notin H$.

Case 2: a block is completely deleted. Then $d_j = p$ for that block. Since the total deletion is at most p, all other d_ℓ are zero. Depending on which block is deleted, the result is one of

$$1^p 0^p 1^p, \qquad 0^{2p} 1^p, \qquad 0^p 1^{2p}, \qquad 0^p 1^p 0^p.$$

Each string contains both symbols but has fewer than four maximal blocks. Every string in H containing both symbols has $m, n > 0$ and therefore exactly four maximal blocks: 0^m, 1^n, 0^m, 1^n. Hence none of these pumped-down strings belongs to H.

In every valid decomposition, exponent $i = 0$ produces a string outside H. This contradicts the pumping lemma, which requires the existence of a valid decomposition for which $uv^i xy^i z \in H$ for **every** $i \geq 0$. Therefore H is not context-free.

If D were context-free, then $D \cap R$ would be context-free because CFLs are closed under intersection with regular languages. Since H is not context-free, D is not context-free.

4.3 Optional Tock checks

Tock can construct and parse CFGs directly, so no local CYK or PDA simulator is needed. These finite runs support the examples; they do not replace the general pumping and closure proofs above.

[ ]:
```
# Install once in a fresh environment, then import Tock.
# %pip install -q tock
from tock import *

# The CNF grammar from Problem 2 (with S0 as the start variable).
cnf_anbn = Grammar.from_lines([
    "S0 -> U0 T",
    "S0 -> U0 U1",
    "S -> U0 T",
    "S -> U0 U1",
    "T -> S U1",
    "U0 -> 0",
    "U1 -> 1",
])
display(cnf_anbn)

for word in ["0 1", "0 0 1 1", "0 0 0 1 1 1", "0 0 1", "0 1 0 1"]:
    try:
        tree = any_parse(cnf_anbn, word)
    except ValueError:
        print(f"{word!r:18} reject")
    else:
        print(f"{word!r:18} accept")
        display(tree)
```

[ ]:
```
# Problem 4 is context-free, so construct its grammar and let Tock parse␣
↪examples.
anbn_cm = Grammar.from_lines([
    "S -> A C",
    "A -> a A b",
    "A -> &",
    "C -> c C",
    "C -> &",
])

for word in ["&", "a b", "a a b b c c", "a b b", "a b c c c"]:
    try:
        tree = any_parse(anbn_cm, word)
```

```
    except ValueError:
        print(f"{word!r:18} reject")
    else:
        print(f"{word!r:18} accept")
        display(tree)
```

4.4 What to take away

- CNF separates terminals from binary variable combinations and removes unit rules.
- The CFL pumping lemma pumps two pieces together. The short window vxy makes block-based case analysis possible.
- A grammar or PDA is often easier than pumping when the language is context-free.
- CFLs are closed under union, concatenation, star, intersection with regular languages, and homomorphism.
- CFLs are not closed under unrestricted intersection or complement.
- Regular filters and homomorphisms can expose a known non-context-free core inside a more complicated language.